Expert Systems With Applications 273 (2025) 126822

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

SDELP-DDPG: Stochastic differential equations with Lévy processes-driven deep deterministic policy gradient for portfolio management

Zhen Huang [a], Junwei Duan [b],*, Chuanlin Zhang [a], Wenyong Gong [a],*

[a] Department of Mathematics, Jinan University, Guangzhou, China
[b] Faculty of Data Science, City university of Macau, Macau

ARTICLE INFO

Keywords:
Portfolio management
Stochastic differential equation
Deep reinforcement learning
Convolutional neural network
Lévy process

ABSTRACT

Portfolio management (PM) involves the ongoing redistribution of funds among various financial products, aiming to seek a balance between returns and risks. In this paper, we propose SDELP-DDPG, a novel approach to portfolio management that combines stochastic differential equations (SDEs) with Lévy processes and the deep deterministic policy gradient (DDPG) technique. To alleviate the challenges posed by exploration limitations and enhance the stability of DDPG, we employ SDEs driven by Lévy processes, with drift and diffusion coefficients represented by convolutional neural networks, to generate action policies. Additionally, we devise a reward function, which considers relative entropy, to guide RL agents in learning imitation policies using DDPG. Moreover, we incorporate an attention mechanism and the Ornstein–Uhlenbeck process to choose optimal actions. Our proposed algorithm is evaluated on three real-world datasets: the Dow Jones Industrial Average markets, the Energy markets and the cryptocurrency markets, and the experimental results validate the effectiveness of SDELP-DDPG compared to existing PM approaches.

1. Introduction

The overall goal of financial investment is to dynamically allocate a set of assets to maximize long-term returns while minimizing risks. Since the first portfolio optimization model was proposed by Markowitz (Markowitz, 1952), an inevitable task is to estimate the optimal portfolio weights of financial models in pursuit of long-term stable returns and low risks. However, modeling real-world financial market behavior encounters challenges like non-stationarity, poor predictive behavior, and weak historical coupling. These issues have received particular attention in the quantitative financial community, leading to ongoing efforts to address them using various methods (Lai, Dai, Ren, & Huang, 2017; Lai, Tan, Wu, & Fang, 2020; Li & Hoi, 2012). Unfortunately, traditional mathematical and statistical methods suffer from the restricted assumptions and flexibility of mathematical models, resulting in the failure of the optimal balance between expected returns and risks.

To address these challenges, many machine learning-based portfolio management algorithms have been developed to design optimal reallocation strategies (Almahdi & Yang, 2017; Gunjan & Bhattacharyya, 2023; Isichenko, 2021; Li, Wu, Lu, Wang, & Hu, 2020; Ye et al., 2020), due to their powerful approximation ability and outstanding estimation performance. Among the machine learning methods, reinforcement learning (RL) demonstrates significant potential in many financial applications, such as algorithmic trading (Huang, Gong, & Duan, 2023; Huang, Li, Mei & Gong, 2023) and portfolio management (Wang, Huang, Tu, Zhang, & Xu, 2021; Ye et al., 2020). Similar to the cognitive process in humans, reinforcement learning allows agents to interact with unfamiliar environments and learn complex rules. However, traditional RL methods, such as Q-learning, SARSA and policy gradient, suffer from the limitation of representation ability. Therefore, various deep neural networks (DNNs), such as recurrent neural networks (RNN) (Medsker & Jain, 2001) and long short-term memory (LSTM) (Memory, 2010), are employed to extract features from time series trading data, enabling reinforcement learning to acquire mappings from states and actions to rewards. This procedure contributes to the development of deep reinforcement learning (DRL), thereby improving the sequential decision-making capability of RL. Therefore, DRL is a promising tool for portfolio management (Almahdi & Yang, 2017; Filos, 2019).

Deep reinforcement learning faces the fundamental challenge of instability, which is related to the balance between exploration and exploitation. One challenge in reinforcement learning is to minimize

* Corresponding authors.
E-mail addresses: hz222546@stu2021.jnu.edu.cn (Z. Huang), jwduan@cityu.edu.mo (J. Duan), tclzhang@jnu.edu.cn (C. Zhang), gongwenyong@jnu.edu.cn (W. Gong).

https://doi.org/10.1016/j.eswa.2025.126822
Received 21 April 2024; Received in revised form 31 January 2025; Accepted 7 February 2025
Available online 17 February 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

algorithmic instability while also encouraging agent exploration during training. Exploitation refers to the agent's choice of the best action with the highest average reward. Nevertheless, one must engage in exploration mode to find out the best action. However, commonly used exploration strategies (e.g., ϵ-greedy and Boltzmann exploration (Sutton & Barto, 2018)) may not adequately explore new possibilities. For example, the ϵ-greedy method cannot continue to explore, and the possibility of deviating from the default trajectory by more than a few steps is extremely small. Due to the exploration operation randomly selecting an action, regardless of the reward value previously obtained, it remains an open problem (Dabney, Ostrovski, & Barreto, 2020). Therefore, a large-area random exploration strategy with stability for DRL is desired to alleviate the issue.

Stochastic differential equations (SDEs) with Lévy processes are a fundamental tool for modeling the complex and random dynamic behavior of financial markets. In SDEs, the drift term models the expected return of an asset, and the volatility term captures the uncertainty or variability around the return. For example, Topaloglou, Vladimirou, and Zenios (2008) focus on portfolio optimization in a stochastic environment using SDEs, which provides important insights into incorporating uncertainty in portfolio management. Unlike Brownian motion, Lévy processes can account for discontinuous jumps for encouraging exploration, which are seen in real markets (e.g., stock market crashes or unexpected corporate news). Moreover, the Lévy-Khintchine theorem (Ken-Iti, 1999) states that any Lévy process can be decomposed into the sum of a Brownian motion with drift and an independent Lévy jump process. This decomposition allows for a more general and realistic representation of asset returns, capturing both continuous dynamics and sudden jumps. Therefore, SDEs with Lévy processes are desired in portfolio management.

Portfolio management, as a continuous decision-making optimization problem, aims to find the optimal asset weights to maximize returns while minimizing risks. To address this challenge using Deep Reinforcement Learning (DRL), a continuous framework is essential. Additionally, previous studies have shown that incorporating SDEs into reinforcement learning can enhance DRL performance. For instance, Chen, Cheng, Liu, Jia, and Ma (2019) proposed an IDL method based on SDEs with Brownian motions, demonstrating improved performance in DRL tasks. More recently, Yang, Gao, Lu, Duan, and Liu (2023) introduced LDE-Net, which uses neural SDEs with Lévy motion to predict chaotic time series trends. Compared to other continuous reinforcement learning methods (e.g., RDPG (Song, Yang, McGreavy, & Li, 2018)) that face difficulties in generalizing to entirely new settings or dealing with unexpected situations, DDPG framework (Lillicrap et al., 2015) can naturally integrate with SDEs, effectively mitigating the aforementioned issues and meeting portfolio management requirements. Based on these insights, we propose an action generator $\mathbf{a}^t$ within the DDPG framework, based on SDE with Lévy motion: $d\mathbf{a}^t = u\left(\mathbf{a}^t;\theta^u\right)dt + \vartheta\left(\mathbf{a}^t;\theta^\vartheta\right)dL_\alpha^t$, where u is the drift term, ϑ is the diffusion term, and L_α^t is a symmetric α-stable Lévy process in $\mathcal{R}^n$. Therefore, DDPG equipped with Lévy SDEs is highly suitable for portfolio management tasks.

In this paper, SDEs with Lévy processes driven deep deterministic policy gradient (SDELP-DDPG) is proposed for portfolio management, which can effectively address the efficient problem of the portfolio model and adequately balance the exploitation and exploration issues of reinforcement learning. By the aid of the DDPG algorithm, we have developed an objective reward function that balances return and risk, and have also incorporated SDEs and multi-layer residual networks to model action policy generators and critic functions. The action policy generator introduces a stochastic system induced by Lévy motion, which is used in the financial field. Furthermore, the Ornstein–Uhlenbeck (OU) process is applied to determine the optimal action.

The main contributions of this paper are concluded as follows:

- An objective reward function linked to relative entropy is proposed for balancing portfolio return and risk.
- We present an SDELP-DDPG framework for portfolio management, designed to handle market fluctuations and balance between exploitation and exploration. The actor policy generator of SDELP-DDPG is simulated by SDEs equipped with CNN and α-stable Lévy process.
- We provide an optimal action selection strategy by incorporating the attention mechanism and the Ornstein–Uhlenbeck process.
- Various experiments, including the statistical significance, are performed, and our method outperforms other methods on the criteria, such as Annual Return and Cumulative Returns Ratio, which shows the effectiveness of Lévy stochastic strategy and the proposed SDELP-DDPG method.

The rest of the paper is organized as follows. In the next section, related work is reviewed. In Section 3, the portfolio management problem is formalized with reinforcement learning, and the action policy generator is also presented. In Section 4, experimental results are shown. Finally, conclusions and future work for portfolio management are discussed.

2. Related work

In this section, we summarize the most relevant work and discuss its relationship to the proposed work.

2.1. Portfolio management

Portfolio management has caught significant attention from both the academic and investment communities. The goal is to find the optimal investment strategy that allows investors to balance returns and risks by allocating across various financial instruments without prior knowledge of market outcomes. In fact, the underlying model of portfolio management requires a stochastic simulation of market behavior involving stock prices (such as the random walk or Ornstein–Uhlenbeck random process (Renshaw, 1987)). Portfolio management is the decision-making process of continuously reallocating an amount of funds into a number of different financial investment products, aiming to maximize the returns while restraining the risks (Markovitz, 1959). The mean–variance strategy (Kolm, Tütüncü, & Fabozzi, 2014) is to choose a portfolio that strikes a balance between risks (variance) and expected returns (mean) across a group of assets. The Capital Asset Pricing Model (CAPM) (Lintner, 1975; Mossin, 1966; Treynor, 1961) is an essential tool in the field of portfolio management. The model aids investment professionals in assessing the potential returns of an investment, specifically for individual stocks or collections of stocks within a portfolio. The Black-Litterman model (Black & Litterman, 1992) is designed for the purpose of optimizing asset allocation within a portfolio by taking into consideration both the investor's personal opinions and the market's equilibrium returns. Factor-based portfolio optimization (Auh & Cho, 2023) enhances CAPM by systematically incorporating factors known to impact asset returns. Risk-Parity is an equal risk distribution strategy that ignores investor views, and it has been widely used since the Global Financial Crisis in 2008 (Costa & Kwon, 2019; Roncalli & Weisang, 2016). Stochastic Portfolio Theory (SPT) (Fernholz & Fernholz, 2002) is a framework that constructs portfolios based on relatively strong model assumptions. In addition, traditional portfolio management methods also contain the following strategies: Buy and Hold, Best Stock Constant Rebalanced Portfolio (BCRP), Follow-the-Winner (Cover & Ordentlich, 1996), Follow-the-Loser (Li, Zhao, Hoi, & Gopalkrishnan, 2012), and Pattern-Matching-Based Approaches (Györfi, Lugosi, & Udina, 2006).

More recently, machine learning methods have been effectively used to develop portfolio strategies due to their capability for nonlinear mapping and feature representation. The deep learning hierarchical

model (Heaton, Polson, & Witte, 2017) is used to construct portfolios and produce more convincing results than traditional methods. On the other hand, deep reinforcement learning (DRL) methods combine the perceptual ability of deep learning with the decision-making ability of RL, and thus offer strong versatility and direct control from input to output. The end-to-end learning system has achieved great success in dealing with complex tasks. DRL is therefore thought to be a promising candidate for solving the portfolio problem. Jiang, Xu, and Liang (2017) present a financial model-free reinforcement learning framework to provide a deep machine learning solution for portfolios. DRL also demonstrates the capability to address a wide range of high-dimensional issues in algorithmic trading (Huang, Gong, & Duan, 2023; Huang, Li, et al., 2023).

In these works, the solution of the policy function plays an important role in RL algorithms. Given the system's inherent nonlinearity and stochastic nature, tackling this issue poses a significant challenge for conventional control techniques (Zhu, 2014). Inspired by the work (Liang, Chen, Zhu, Jiang, & Li, 2018) that models RL with CNN for portfolio management, we attempt to approximate the drift and diffusion coefficients of SDEs using convolutional neural networks, and use Deep Deterministic Policy Gradient (Lillicrap et al., 2015) to train the RL agent.

2.2. Stochastic differential equations

Chen et al. (2019) note that conventional RL methods struggle to balance the uncertainties arising from continuity and variance control, and stochastic differential equations (SDEs) can alleviate the issue. For financial market systems, SDEs can be used to quantify the set of optimal pricing barriers (Han, Hu, & Lee, 2016). Originating from mathematical finance, Sharp (1990) use SDEs to aid in the analysis of problems in finance. From the perspective of SDE, market price fluctuations are often seen as the "derivative" of Brownian motion (Huang, Wang, & Yi, 2017).

Financial markets often show sharp price movements (known as jumps), especially during times of high volatility or market crises. The Lévy process is highly valuable in financial modeling and portfolio management since it can capture aspects of real financial markets. Merton (1976) introduces the jump-diffusion model combining Brownian motion and Poisson processes to account for sudden jumps, illustrating the importance of Lévy processes in financial modeling. Becker, Clements, and McClelland (2009) examines the use of jump-diffusion models (a form of Lévy process) for modeling equity index returns, showing how jumps can significantly improve the fit of asset price models. In many real-world scientific and engineering applications (Naseri & Malek, 2014), Lévy motion finds extensive use in option pricing models (Yuan, Ding, Duan, Lu, & Wu, 2022) and agricultural product pricing (Kurumatani, 2020). In these paper, we employ the α-stable Lévy motion to capture uncertainty for encouraging exploration.

3. Methodology

3.1. Problem setting

A stock trader invests in a set of assets and is allowed to reallocate in order to maximize his profit over a period of time. The market is assumed to be continuous, meaning that the closing price corresponds to the opening price of the following day. A trading agent monitors the exchange every day, analyzes the data and then reallocates their portfolio. Furthermore, we assume that the agent makes a reallocation at the end of the trading day, which means that all reallocations can be made at the states.

Consider a portfolio containing n stocks. The closing price of the ith asset after a period t is $x^t_{i,close}$. The closing prices of all assets comprise the price vector for a period t as $\mathbf{x}^t_{close}$. The performance of the stocks can be described by a vector of relative prices (Jiang et al., 2017), denoted by $\mathbf{v}^t = (v^t_1, v^t_2, \dots, v^t_n)^\top$, where v^t_i is the next day's opening price of the ith stock divided by its opening price on the current day. That is,

$$\mathbf{v}^t := \frac{\mathbf{x}^t}{\mathbf{x}^{t-1}} = \left(1, \frac{x^t_1}{x^{t-1}_1}, \dots, \frac{x^t_n}{x^{t-1}_n}\right)^\top. \tag{1}$$

In addition, $\mathbf{w}^t = (w^t_0, w^t_1, \dots, w^t_n)$ is used to represent the portfolio weights at the beginning of the tth period with constraint $\sum_{i=1}^n w^t_i = 1$, where w^t_i is the weight of the asset i at the beginning of the tth period.

Noting that in a Markov decision process, most objective functions take the form $R = \sum_{t=1}^T \gamma^t r(\mathbf{v}^t, \mathbf{a}^t)$, where $\mathbf{v}, \mathbf{a}, r, \gamma$ represent the state, action, reward and discount rate, respectively.

Generally speaking, a state $\mathbf{v}$ includes previous open, closing, high, low price, volume or some other financial indexes in a fixed window, and actions $\mathbf{a}$ are the desired allocating weights. Thus, $\mathbf{a}^{t-1} = (a^{t-1}_0, a^{t-1}_1, \dots, a^{t-1}_n)$, subject to the constraint $\sum_{i=1}^n a^{t-1}_i = 1$. $\mathbf{w}^{t-1}$ represents the reallocated weight at the end of time $t-1$. The weight vector from the previous period $(t-1)$ is utilized to construct a portfolio that will be managed by a portfolio manager until the end of the period. As a result of the price fluctuation within a day, the weight vector $\mathbf{a}^{t-1}$ at the beginning of the day would evolve into $\mathbf{w}^{t-1}$ at the end of the day:

$$\mathbf{w}^{t-1} = \frac{\mathbf{a}^{t-1} \odot (\mathbf{v}^{t-1})^\top}{\mathbf{a}^{t-1} \cdot \mathbf{v}^{t-1}}, \tag{2}$$

where $\odot$ represents the element-wise product.

Transaction cost, which is measured as a fraction of transaction amount, has also been taken into considerations in this paper. Transaction cost $u_c(t)$ should be subtracted from the fluctuation of wealth $\mathbf{a}^{t-1} \cdot \mathbf{v}^{t-1}$, and then we get the net return $\mathbf{a}^{t-1} \cdot \mathbf{v}^{t-1} - u_c(t)$. Moody, Wu, Liao, and Saffell (1998) proposed an approximation of $u_c(t) = \varphi \sum_{i=1}^n |w^t_i - w^{t-1}_i|$. However, some researchers pointed out that the cost function will get a relatively large value when the market fluctuates greatly. In order to eliminate the effect, we design a relative transaction cost function as shown below.

$$u_c(t) = \varphi \frac{\mathbf{w}^t - \mathbf{w}^{t-1}}{\mathbf{w}^t \cdot \mathbf{v}^t} \cdot \mathbf{v}^t. \tag{3}$$

Specifically, we set cost rate $\varphi = 0.25\%$, which is an assumption commonly used in practical trading experiences and academic precedents. In order to reflect typical trading costs, many researchers use the same value for modeling. For example, the research Balduzzi and Lynch (1999) examines how transaction costs affect portfolio returns and mentions estimates in the range of 0.25%, and Gârleanu and Pedersen (2013) incorporates transaction costs and assumes values similar to 0.25% for equities in their numerical examples. In summary, we design the immediate reward r_t as:

$$r_t = \log\left(\mathbf{a}^{t-1} \cdot \mathbf{v}^{t-1} - \varphi \frac{\mathbf{w}^t - \mathbf{w}^{t-1}}{\mathbf{w}^t \cdot \mathbf{v}^t} \cdot \mathbf{v}^t\right) \tag{4}$$

The portfolio value P_{t-1} at the beginning of the tth period is affected by the realized asset change during the tth period. Given an initial investment amount P_0 and an initial portfolio vector $\mathbf{w}^0 = (1, 0, \dots, 0)^T$, we can then get the final portfolio value P_t

$$P_t = P_0 \exp\left(\sum_{t=1}^T r_t\right) = P_0 \prod_{t=1}^T \left(\mathbf{a}^{t-1} \cdot \mathbf{v}^{t-1} - \varphi \frac{\mathbf{w}^t - \mathbf{w}^{t-1}}{\mathbf{w}^t \cdot \mathbf{v}^t} \cdot \mathbf{v}^t\right). \tag{5}$$

Due to the limitation of training data, our reinforcement learning agent may underestimate the risk when training in a bull market,[1] which may lead to disastrous deterioration in its performance in a real trading environment. Different approaches in finance can help evaluate

[1] A bull market refers to a period in the financial markets where the prices of assets, such as stocks, bonds, or commodities, are rising or are expected to rise. The term "bull" comes from the way a bull attacks—by thrusting its horns upward, symbolizing the upward movement of the market.

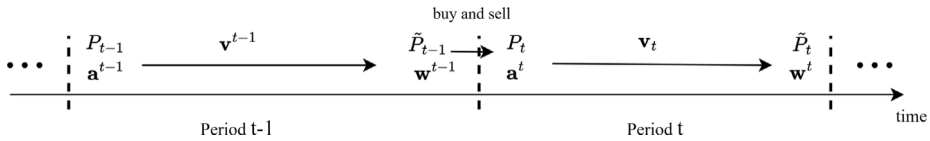

Fig. 1. The evolution of weight vectors and portfolio value.

the current portfolio risk to alleviate the effect of biased training data. In the work Almahdi and Yang (2017) and Helmbold, Schapire, Singer, and Warmuth (1998), a risk-adjusted function is defined as

$$\mathrm{D}(\mathbf{w}^t \parallel \mathbf{w}^{t-1}) \overset{\text{def}}{=} \sum_{i=1}^{n} w_i^t \log(\frac{w_i^t}{w_i^{t-1}}).$$

In essence, the function relative entropy which is commonly used in machine learning to overcome the problem of overfitting. We thus define our objective function as follows:

$$\begin{aligned} R^t &= \gamma^t \left(r_t(\mathbf{v}^t, \mathbf{a}^t) - \beta \mathrm{D}(\mathbf{w}^t \parallel \mathbf{w}^{t-1})\right), \\ R &= \sum_{t=1}^{T} R^t, \end{aligned} \tag{6}$$

where D is the relative entropy that serves as a penalty term to balance risks, and $\beta > 0$ is a weight parameter. The evolution of weight vectors and portfolio value can be seen in Fig. 1.

3.2. Action policy generator

Since many complex real-world phenomena exhibit abrupt, intermittent, or jumping behaviors, such as impulses, using Brownian motion to capture data uncertainty may not be suitable. Lévy processes are actually stochastic processes with stationary and independent increments. The Lévy process can accurately describe certain complex distribution characteristics, such as peaks, thick tails and biases. It can also reflect the discontinuity displayed in the asset movement process, so it has been widely and effectively used in the financial scenarios. Due to the properties of Lévy motion, we choose it to better capture the uncertainty in financial data. Thus, the market price fluctuations are simulated by α-stable Lévy motion, and make it the Euler discretization form of the stochastic differential equation. The action policy generator that predicts financial data and makes investment decisions has demonstrated convergence properties in Yang et al. (2023). We do not consider taking direct actions in the Markov controlled output, and we focus on incremental changes, which can be described by Eq. (7).

$$\begin{aligned} &d\mathbf{a}^t = u\left(\mathbf{a}^t; \theta^u\right) dt + \vartheta\left(\mathbf{a}^t; \theta^\vartheta\right) dL_\alpha^t, \quad \mathbf{a}^t \in \mathcal{R}^n, \\ &\mathbf{a}^0 = (1, 0, \ldots, 0) \end{aligned} \tag{7}$$

where $u : \mathcal{R}^n \to \mathcal{R}^n$ is a drift function, $\vartheta : \mathcal{R}^n \to \mathcal{R}^n \times \mathcal{R}^n$ is a diffusion function, and L_α^t is a symmetric α-stable Lévy process in $\mathcal{R}^n$. θ^u and θ^ϑ are parameters of the two functions. Moreover, both drift and diffusion functions in Eq. (7) are approximated by two different multilayer perceptron (MLP) neural networks, which are called the drift net and diffusion net, respectively. Especially, the Euler–Maruyama scheme with a fixed step size is used to solve the SDE (7). Based on the Euler–Maruyama scheme, the discrete scheme of Eq. (7) is shown in Eq. (8):

$$\begin{aligned} &\mathbf{a}_1^t = conv2d(\mathbf{v}^t) \\ &\mathbf{a}_{k+1}^t = \mathbf{a}_k^t + u\left(\mathbf{a}_k^t; \theta^u\right) \triangle t + \vartheta\left(\mathbf{a}_1^t; \theta^\vartheta\right) (\triangle t)^{1/\alpha} L_k^t \\ &k = 1, \ldots, m, \\ &\mathbf{a}^t = \mathbf{a}_{m+1}^t + attention \end{aligned} \tag{8}$$

where $\mathbf{a}_k^t$ and $\mathbf{v}_k^t$ represent the kth layer of an action a and a state v at time t, and *attention* is an attention network. We in this paper denote Eq. (8) as the action policy generator, which has the following representation.

$$\mathbf{a} = \Phi(\mathbf{v}, L|\theta^u, \theta^\vartheta) \tag{9}$$

where $\Phi : \mathbb{V} \to \mathbb{A}$. Symmetric α-Stable Random Variables $\mathbf{X} \sim S^\alpha(l_1, l_2, l_3)$ is called a symmetric α-stable random variable if $l_2 = 0$ and $l_3 = 0$, that is, $\mathbf{X} \sim S^\alpha(l_1, 0, 0)$. When $l_1 = 1$, it is called a standard symmetric α-stable random variable, and we denote this by $\mathbf{X} \sim S^\alpha(1, 0, 0)$. Now we consider the diffusion as n-step random walk with Lévy noise, here $L^t \sim S^\alpha(1, 0, 0)$ are independent standard symmetric α-stable random variables.

Definition 2.1. Symmetric α−Stable Lévy Motions L_α^t is a stochastic process which satisfies the following properties:(i) $L_\alpha^0 = 0$, a.s. (ii) L_α^t has independent increments. (iii) $L_\alpha^t - L_\alpha^k \sim S^\alpha\left((t-k)^{1/\alpha}, 0, 0\right)$. (iv) L_α^t has stochastically continuous sample paths, that is, for every $k > 0$, $L_\alpha^t \to L_\alpha^k$ in probability, as $t \to k$.

As discussed above, we can quantify uncertainty using Lévy motion. To make the system able to achieve good action prediction, we design our neural SDE model with Lévy motion, which has two separate neural nets to represent the drift function and the diffusion function, and its structure is shown in Fig. 2. The drift net $u\left(\mathbf{a}^t; \theta^u\right)$ aims to control the action strategy and capture aleatoric uncertainty, which is achieved by representing the output as a probabilistic distribution. The drift net u and diffusion net ϑ are devoted to capture the aleatoric and epistemic uncertainty, respectively, to achieve good portfolio.

3.3. Deep reinforcement learning

Deep reinforcement learning has many advantages, including enabling the agent to explore, being an end-to-end intelligent learning process, and preventing the curse of dimensionality. We conduct the deep deterministic policy gradients (DDPG) to extend the applications in portfolio management. DDPG combines Q-learning and policy gradient, effectively using a neural network as its function approximator based on Deterministic Policy Gradient algorithms (Silver et al., 2014).

3.3.1. Q-learning

Q-learning is a reinforcement learning based on Q-value function. To be specific, a Q-value function $Q^\pi(v, a)$ gives the expected accumulated reward when executing an action $\mathbf{a}$ in a state $\mathbf{v}$ and following a policy π in the future, and it has the following formula:

$$Q^\pi(v, a) = \mathbb{E}_{(\mathbf{v}^{i>t}, \mathbf{a}^{i>t})}\left[R^t | \mathbf{v}^t = v, \mathbf{a}^t = a\right].$$

The Bellman Equation allows us to compute it by recursion:

$$Q^\pi(v, a) = \mathbb{E}_{v \sim D}\left[R^t(v, a) + \gamma \mathbb{E}_{a' \sim \pi(\theta)}\left[Q^\pi\left(v', a'\right)\right]\right].$$

For a stochastic policy depicted by Lévy motion Φ in Eq. (9), the above equation can be rewritten as:

$$Q^\Phi(v, a) = \mathbb{E}_{v' \sim D}\left[R^t(v, a) + \gamma\left[Q^\Phi\left(v', \Phi(v')\right)\right]\right]. \tag{10}$$

Q-learning learns the action-value function $Q^\Phi(v, a^*)$ corresponding to the optimal policy by minimizing:

$$\begin{aligned} \mathrm{L}(\theta^Q) &= \mathbb{E}_{v,a,R,v'}\left((Q(v, a|\theta^Q) - y)^2\right) \\ y &= R^t + \gamma \max_{a'} \tilde{Q}(v', a'^* | \theta^{\tilde{Q}}) \\ a'^* &= \arg\max_{a'} Q(v', a' | \theta^Q) \end{aligned} \tag{11}$$

One crucial component of stabilizing Q-learning is the use of an experience replay buffer D containing tuples (v, a, R, v'). Another component to help stabilize learning is a target $\tilde{Q}$ function whose parameters are softly updated with the most recent parameter θ^Q.

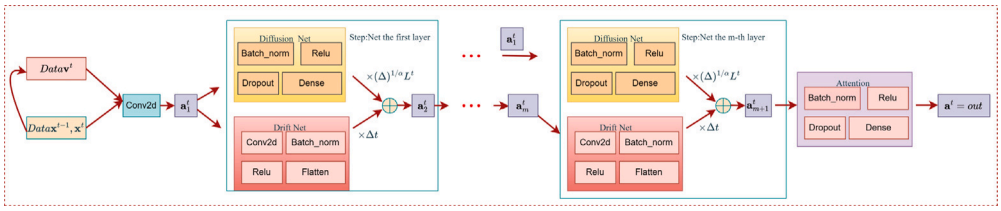

Fig. 2. Structure of the action policy generator at period t.

3.3.2. Deterministic policy gradient (DPG) algorithms

Policy gradient (PG) methods are another popular choice for a variety of RL tasks. The main idea of PG is to directly adjust the parameters θ of the policy Φ in order to maximize the objective.

$$J(\Phi_\theta) = \sum_\tau R(\tau)\Phi_\theta(\tau) = \mathbb{E}_{a\sim\Phi_\theta(\tau)}(R(\tau)) \tag{12}$$

where $R(\tau)$ is already defined in Eq. (6). The gradient of $\nabla J(\Phi_\theta)$ is as follows.

$$\begin{aligned}\nabla J(\Phi_\theta) &= \sum_\tau R(\tau)\nabla\Phi_\theta(\tau)\\ &= \sum_\tau R(\tau)\Phi_\theta(\tau)\frac{\nabla\Phi_\theta(\tau)}{\Phi_\theta(\tau)}\\ &= \sum_\tau R(\tau)\Phi_\theta(\tau)\nabla\log\Phi_\theta(\tau)\\ &= \mathbb{E}_{\tau\sim\Phi_\theta(\tau)}\left[R(\tau)\nabla_\theta\log\Phi_\theta(\tau)\right]\\ &= \mathbb{E}_{\tau\sim\Phi_\theta(\tau)}\left[Q^\Phi(\tau)\nabla_\theta\log\Phi_\theta(\tau)\right]\\ &= \mathbb{E}_{v\sim\mathcal{D},a\sim\Phi_\theta}\left[\nabla_\theta\log\Phi_\theta(v)Q^\Phi(v,a)\right].\end{aligned} \tag{13}$$

The above formula is the gradient calculation formula of the stochastic policy gradient (SPG) based on the Q-value. It can be seen that the stochastic policy gradient needs to be sampled in the entire action space Φ_θ. Compared with the SPG, the deterministic policy gradient (DPG) has less integration for the action and more derivatives of the Q function for the action. The gradient calculation formula of DPG based on Q-value as follows:

$$\nabla J(\Phi_\theta) = \mathbb{E}_{v\sim\mathcal{D}}\left[\nabla_\theta\Phi_\theta(v)\nabla_a Q^\Phi(v,a)|_{a=\Phi_\theta(v)}\right] \tag{14}$$

DPG algorithms extends the policy gradient algorithm to deterministic policies $\Phi_{(\theta^u,\theta^\vartheta)} : \mathbb{V}\to\mathbb{A}$. In other words, the θ mentioned above is the general term for θ^u and θ^ϑ, $\theta : (\theta^u,\theta^\vartheta)$. In deep deterministic policy gradient, six networks are required: online actor (Drift-Net, Diffusion-Net), online critic, target actor (target Drift-Net, target Diffusion-Net) and target critic. Combining Q-learning and policy gradient, actor is the stochastic differential function Φ and critic is the Q-value function. The agent observes a state, and the actor would provide an "optimal" action in the continuous action space. Then the online critic would evaluate the actor's action and update the online actor. What is more, the target actor and the target critic are used to update the online critic. In particular, under certain conditions, we can write the gradient of the objective in Eq. (12) as follows:

$$\begin{aligned}&\nabla_\theta J = \mathbb{E}_{v\sim\mathcal{D}}\left[\nabla_\theta Q^\Phi(v,a|\theta^Q)|_{a=\Phi(v^t,L^t|\theta^u,\theta^\vartheta)}\right]\\ =&\mathbb{E}_{v\sim\mathcal{D}}\left[\nabla_\theta\Phi(\cdot)\nabla_a Q^\Phi(v,a|\theta^Q)|_{a=\Phi(v^t,L^t|\theta^u,\theta^\vartheta)}\right]\\ =&\mathbb{E}_{v\sim\mathcal{D}}\left\{\nabla_a Q^\Phi(v,a|\theta^Q)|_{a^t=\Phi(v^t,L^t|\theta^u,\theta^\vartheta)}\right.\\ &\left.\left[\nabla_{\theta^u}\Phi(\cdot)+\nabla_{\theta^\vartheta}\Phi(\cdot)\right]|_{v=v^t}\right\}\end{aligned} \tag{15}$$

The above formula is derived by the chain rule $\frac{\partial Q}{\partial\theta} = \frac{\partial Q}{\partial a}\cdot\frac{\partial a}{\partial\theta} = \frac{\partial Q}{\partial a}\cdot(\frac{\partial a}{\partial\theta^u}+\frac{\partial a}{\partial\theta^\vartheta})$. Deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015) is a variant of DPG where the policy Φ and critic Q^Φ are approximated with deep neural networks. The Q-value function is approximated by a multi-layer convolutional residual neural network, while the actor strategy is modeled using a neural SDE with Lévy motion. The detailed architecture of the Actor policy and the Critic is presented in Fig. 2 and Fig. 3, respectively.

In competitive environments, the learned policies can capture epistemic uncertainty. It is known that a crucial trade-off in reinforcement learning is between exploration and exploitation. The Ornstein–Uhlenbeck random process is an efficient adversarial training technique, which allows the agent to explore environments with momentum properties very well. The Ornstein–Uhlenbeck random process is highly efficient for exploration. The DDPG framework of the proposed method is shown in Fig. 4. For the purpose of learning portfolio policies, we propose to update policies by using the Ornstein–Uhlenbeck process. We can rewrite Eq. (13) into the following form at time t:

$$\begin{aligned}&\nabla_\theta J = \mathbb{E}_{v\sim\mathbb{D}}\nabla_a Q^\Phi(v,a^*|\theta^Q)|_{a=\Phi(v^t,L^t|\theta^u,\theta^\vartheta)}\cdot\\ &\qquad\left[\nabla_{\theta^u}\Phi(\cdot)+\nabla_{\theta^\vartheta}\Phi(\cdot)\right]|_{v=v^t}\\ &a_1 = conv2d(v)\\ &a_{k+1} = a_k + u\left(a_k;\theta_u\right)\triangle t + \vartheta\left(a_1;\theta_\vartheta\right)(\triangle t)^{1/\alpha}L_k\\ &k = 1,\dots,m\\ &a = a_{m+1} + attention\\ &a^* = \frac{a+\mathfrak{N}}{\sum_j a_j}, j = 1,\dots,n\end{aligned} \tag{16}$$

and

$$\begin{aligned}&\mathrm{L}(\theta^Q) = \mathbb{E}_{v,a,R,v'}\left((Q(v,a)-y)^2\right)\\ &y = R^t + \gamma\max_{a'}\tilde{Q}(v',a'^*)\\ &a' = \Phi(v',L|\theta^u,\theta^\vartheta)\\ &a'^* = \frac{a'+\mathfrak{N}}{\sum_j a'_j}, j = 1,\dots,n\end{aligned} \tag{17}$$

where $\mathfrak{N}$ represents the Ornstein–Uhlenbeck random process (called OU process). Finally, combining Eqs. (16) and (17) completes the SDELP-DDPG algorithm, which is summarized as Algorithm 1.

3.3.3. Dynamic portfolio optimization and reinforcement learning

Researchers Chaouki, Hardiman, Schmidt, Sérié, and De Lataillade (2020) and Quer and Borrell (2022) connect optimal control problems to reinforcement learning because both share the same underlying framework, i.e., the Markov Decision Process (MDP). Formally, the RL problem is a (stochastic) control problem of the following form:

$$\max_{\{a^t\}}\mathbb{E}\left[\sum_{t=0}^{T-1}\gamma^t r^t - \text{cost}(u) - \text{risk}(\mathrm{D})\right].$$
$$\text{s.t. constraints}$$

Taking utility maximization into account, the above formula can be brought into the following form:

$$\begin{aligned}&\max_{\{a^t\}}\mathbb{E}[\sum_{t=0}^{T-1}\gamma^t\log\left(\mathbf{a}^{t-1}\cdot\mathbf{v}^{t-1} - \varphi\frac{\mathbf{w}^t-\mathbf{w}^{t-1}}{\mathbf{w}^t\cdot\mathbf{v}^t}\cdot\mathbf{v}^{\mathbf{t}}\right) -\\ &\qquad\beta\mathrm{D}(\mathbf{w}^t\parallel\mathbf{w}^{t-1})]\\ &\text{s.t.}\begin{cases}d\mathbf{a}^t = u\left(\mathbf{a}^t;\theta^u\right)dt + \vartheta\left(\mathbf{a}^t;\theta^\vartheta\right)dL^t_\alpha, \mathbf{a}^t\in\mathcal{R}^n\\ v^t\leftarrow\text{sample}\,\mathcal{M}(v^{t-1})\\ \mathbf{w}^{t-1} = \frac{\mathbf{a}^{t-1}\odot(\mathbf{v}^{t-1})^\top}{\mathbf{a}^{t-1}\cdot\mathbf{v}^{t-1}}\qquad 0\le t\le T-1\\ \mathbf{a}^0 = (1,0,\dots,0)\end{cases}\end{aligned} \tag{18}$$

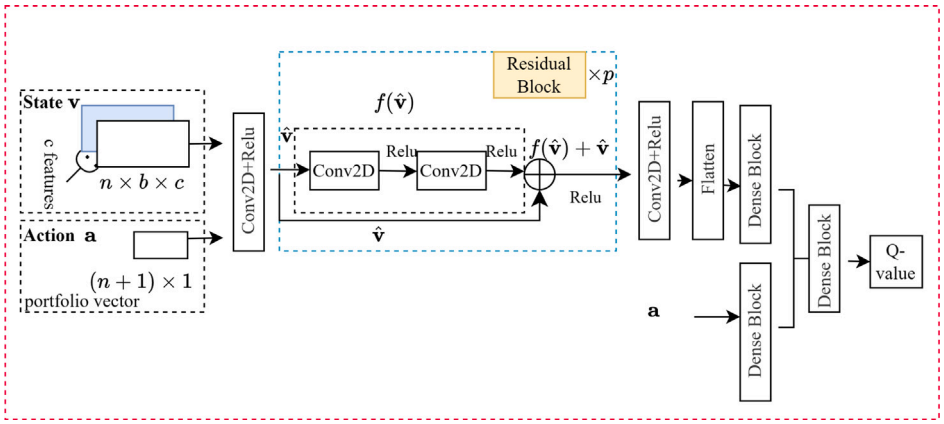

Fig. 3. Multi-layer Convolutional Residual Neural implementation of the Critic Net. The input to the network are an actor tensor and a price tensor, comprising the highest, closing, open or lowest prices of n assets over the past b windows. Then it repeats p times. The outputs are the Q-values.

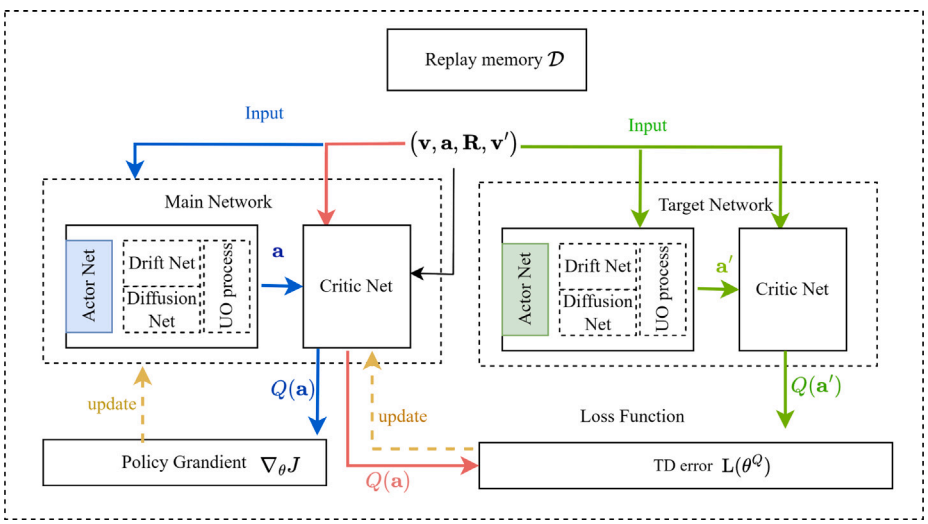

Fig. 4. DDPG framework of the proposed method.

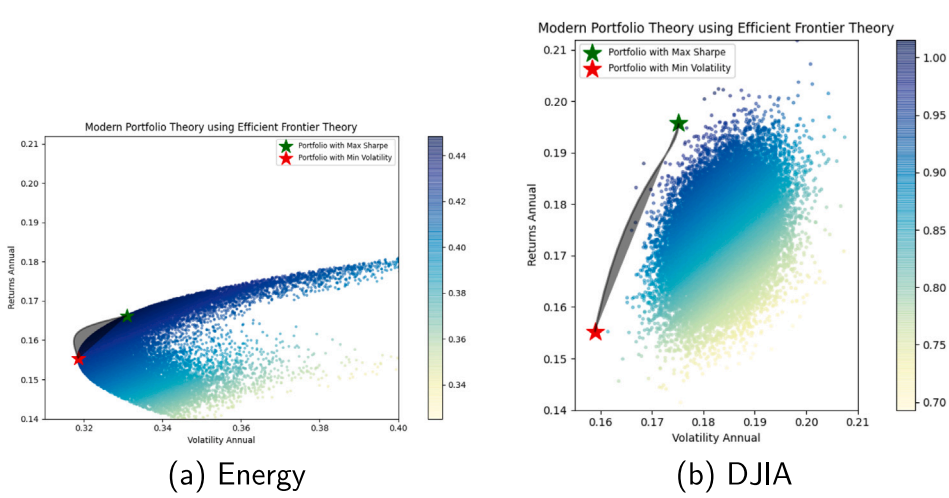

Fig. 5. Efficient Frontier plot for the Energy market (left) and the Dow Jones Industrial Average market (right).

Algorithm 1 Deep Deterministic Policy Gradient for our algorithm (SDELP-DDPG).

Initialize the actor network Φ parameters θ^u and θ^ϑ and the critic network $Q(v, a|\theta^Q)$ and replay buffer $\mathcal{D}$
Create $\tilde{Q}$ and $\tilde{\Phi}$ by $\theta^{\tilde{Q}} \leftarrow \theta^Q$, $\theta^{\tilde{u}} \leftarrow \theta^u$, $\theta^{\tilde{\vartheta}} \leftarrow \theta^\vartheta$
for $epoch = 1, \cdots, N$ **do**
  Acquire the initial observation v^0, OU process $\mathfrak{N}$.
  Set the parameter α of the Lévy motion.
  for $t = 1, \cdots, T$ **do**
    for $layersk = 1, \cdots, m$ **do**
      $a_1^t = conv2d(v^t)$
      Select an action
      $$a_{k+1}^t = a_k^t + u\left(a_k^t; \theta_u\right) \triangle t + \vartheta\left(a_1^t; \theta_\vartheta\right)(\triangle t)^{1/\alpha} L_k^t$$
    end for
    $a^t = a_{m+1}^t + attention$
    Get $a^{t^*} = \frac{a^t + \mathfrak{N}^t}{\sum_j a_j^t}$
    Execute action a^t and observe R^t and $v^{t'}$
    Store both transition in memory buffer $\mathcal{D}$
    Sample a minibatch of $\mathcal{M}$ transitions $(v^i, a^i, R^i, v^{i'})$
    Set $y^i = R^i +$
    $\gamma \max_{a^{i'}} \tilde{Q}(v^{i'}, a^{i'*})|_{a^{i'} = \Phi(v^i, L^i|\theta^u, \theta^\vartheta), a^{i'*} = \frac{a^{i'} + \mathfrak{N}^i}{\sum_j a_j^{i'}}}$
    Update critic by minimizing the loss:
    $\mathrm{L}(\theta^Q) = \frac{1}{\mathcal{M}} \sum_i \left((Q(v^i, a^i|\theta^Q) - y^i)^2\right)$
    Update actor policy by policy gradient:
    $\nabla_{\theta^\Phi} J \approx \frac{1}{\mathcal{M}} \sum_i \nabla_a Q(v^i, a^{i*}|\theta^Q) \cdot$
    $\left[\nabla_{\theta^u} \Phi(\cdot) + \nabla_{\theta^\vartheta} \Phi(\cdot)\right]_{a^i = \Phi(\cdot), a^{i*} = \frac{a^i + \mathfrak{N}^i}{\sum_j a_j^i}}$
    Update the target networks:
    $$\theta^{\tilde{Q}} \leftarrow \varphi\theta^Q + (1-\varphi)\theta^{\tilde{Q}}$$
    $$\theta^{\tilde{u}} \leftarrow \varphi\theta^u + (1-\varphi)\theta^{\tilde{u}}$$
    $$\theta^{\tilde{\vartheta}} \leftarrow \varphi\theta^\vartheta + (1-\varphi)\theta^{\tilde{\vartheta}}$$
  end for
end for

where we additionally assumed that v^t follows a Markovian dynamic. We formulate a dynamic portfolio optimization problems for optimal control policies, and test DDPG against the strategy in the experiments.

4. Experiment and performance evaluation

In this section, we evaluate the SDELP-DDPG trading strategy using performance assessment methodology. We then compare our model's performance on various databases with the same evaluation measurements to benchmark trading strategies.

4.1. Data description and preprocessing

The Dow Jones Industrial Average (DJIA), available from GitHub,[2] represents highly liquid stocks, covering 30 large-cap[3] U.S. companies across various industries, and has been extensively studied and modeled in financial research (Fabbri & Moro, 2018; Hou, Wang, Zhong, & Wei, 2021; Yang, Liu, Zhong, & Walid, 2020). The assets of DJIA markets can be found in Table A.4 in Appendices. The Energy markets, including Brent Oil, Crude Oil WTI, Heating Oil and Natural Gas, have data obtained from Kaggle.[4] Energy markets are highly volatile due to geopolitical factors, supply–demand dynamics, and regulatory changes, therefore they are a typical testing dataset. The data used in this paper spans from 2000 to 2022, encompassing all historical daily energy market data within that period. The corresponding data period is presented in Table A.6 in Section Appendices. Hence, DJIA data and the Energy market data are good choices to verify the effectiveness of the proposed method. In addition, we also conduct experiments using the Cryptocurrency dataset with high volatility and subject to sudden events, whose data was sourced from the website.[5], and some of the top cryptocurrencies ranked by market capitalization can be found in Table A.5 in Appendices.

Prior to inputting data into our proposed network, we utilize a mean value imputation strategy to handle missing data, substituting the gaps with the mean of adjacent data entries. Subsequently, the data undergoes transformation via Eq. (1) to achieve a more compact representation, which is then supplied to the network.

4.2. Evaluation metrics

In this paper, six evaluation metrics (annual return, cumulative return ratio, annual volatility, Sharpe ratio, maximum drawdown and Sortino ratio) are used to measure the performance of portfolio algorithms. The annual return and Sharpe ratio are calculated as follows:

$$\begin{aligned} Annual\ return &= \left(\left(1 + R_E\right)^{365/y} - 1.0\right) \times 100\% \\ Sharpe\ ratio &= \frac{E\left[R_e - R_f\right]}{\sigma} \end{aligned} \tag{19}$$

where R_E and y are the investment income ratio and trading period, and R_f is the risk-free interest rate. In our experiments, E_I is set to \$1000. Sharpe ratio is a measure of the risk-adjusted return. It measures the excess return per unit of deviation in the investment equity, and is typically used as a risk measurement. Cumulative return ratio is the average percentage of accumulative portfolio returns per two years. Maximum Drawdown (MDD), i.e., the maximum observed loss from a peak to a trough of a portfolio before a new peak is attained, is defined as follows.

$$MDD = \frac{PeakValue - TroughValue}{PeakValue}$$

The Sortino ratio is similar to the Sharpe ratio, and the difference is that the former uses downside deviation or downside risk instead of standard deviation as the measure of risk.

4.3. Results

In the subsection, we first analyze the real world data through modern portfolio theory to find the efficient frontier, and then the model's performance under various scenarios is compared with several methods.

4.3.1. Modern portfolio theory

Modern Portfolio Theory (MPT) is an investment theory that revolves around the concept of risk-return trade-off, meaning the higher the potential return, the higher the risk. As an investor, by finding the optimal asset portfolio proportion, it is possible to achieve lower risk. In MPT, random weight vectors are simulated for the portfolio. In order to draw the efficient frontier, 30000 random weight vectors were generated.

We plot the randomly generated portfolios using a color map based on the Sharpe Ratio. As shown in Fig. 5, a curved boundary is formed

[2] https://github.com/Musonda2day

[3] Large-cap (short for "large capitalization") refers to companies with a large market capitalization, which is the total market value of a company's outstanding shares of stock.

[4] https://www.kaggle.com/datasets/prasertk/historical-daily-oil-and-natural-gas-prices?resource=download

[5] https://tianchi.aliyun.com/dataset/144732/

Fig. 6. Comparison of portfolio value P_t with different feature combinations on the Dow Jones Industrial Average market (time: 10/02/2016 ~ 12/29/2020).

by the darkest blue dots towards the left of the cluster obtained. The boundary is called the Efficient Frontier. The green star in the plot represents the portfolio with minimum risk (volatility) and the red star represents the portfolio with maximum Sharpe Ratio. Kanwar et al. mentioned in the paper Kanwar (2019) that the bluer the dot representing the portfolio, the higher the Sharpe Ratio for that portfolio. Therefore, if we offer two investment portfolios with equal returns to risk-averse investors, they will choose the one with the bluest dot, which represents the one with the lowest risk.

4.3.2. Feature combination

It is known that portfolio management generally encompasses abundant features. Common features include OHLCV which refers to the Open-High-Low-Close-Volume price. Therefore, we conduct four experiments under four combinations of features: 1. only with closing prices; 2. with closing and high prices; 3. with closing and open prices; 4. with closing and low prices. As shown in Fig. 6, the results indicate that feature combinations are highly important for the portfolio, and the experiments show that the single closing price could help agents gain the best performance.

4.3.3. Risk

Reward engineering is one of the core topics in designing reinforcement learning algorithms (Dewey, 2014). We design a reward function that uses the cross entropy as a risk measurement, as demonstrated in Eq. (6). The proposed network model is trained for 100 epochs on the DJIA stock market and the energy stock market, respectively. The concept of an optimal portfolio that balances risk and return runs throughout, and we have tested the influence of different β in Eq. (6). As shown in Fig. 7, the reward does not always increase while reducing the penalty factor β, and we can see the reward value achieves the maximum when β is 0.05 on the DJIA market. The fact aligns with Markowitz's balanced risk theory (Markowitz, 1952) that the risk factor in an investment portfolio should neither be too large nor too small, in order to achieve a balance between potential returns and risks. The idea has been a core theme in the development of portfolio theory and modern investment strategies.

4.3.4. Ornstein–Uhlenbeck process

We conduct two groups of experiments. The first group does not contain the Ornstein–Uhlenbeck (OU) process, and the second includes the OU process. The experiments are repeated 50 times, and 25 of them on the Energy market are shown in Table A.7, and the other 25 statistical results for the DJIA market can be found in Table A.8 in Section Appendices. Based on the statistical results, we compute the statistical significance of the annual net profit (ANP) and the Sharpe ratio. The first null hypothesis is:

$$H_0: \quad ANP_1 > ANP_2$$

Table 1
The results of different α. Up: DJIA market. Bottom: Energy market.

Value of α	Annual return	Skewness	Kurtosis	Variation
1.2	0.1870	0.8056	−0.6587	0.3770
1.3	0.1713	0.2782	−0.9479	0.3111
1.4	0.1980	0.4336	−0.5242	0.3749
1.5	0.0826	0.3220	0.2932	0.1283
1.6	0.1067	0.3052	−0.2022	0.3005
1.7	**0.1984**	0.5267	−0.7177	0.3759
1.2	0.1656	0.5348	1.6000	0.1530
1.3	0.1320	0.0344	0.3549	0.1526
1.4	**0.1641**	0.2352	−0.7003	0.2250
1.5	0.1383	−0.0558	−0.5129	0.1966
1.6	0.0519	1.0359	0.6858	0.1596
1.7	0.1543	0.6404	2.2213	0.1668

where ANP_1 represents the model that does not use the OU process, and ANP_2 represents the opposite.

The *p-value* of the *t-test* on the Energy market and DJIA are 2.725454 38 $\times 10^{-14} < 0.01$, $0.0151 < 0.016$, respectively, which means that the null hypothesis is rejected. Therefore, we can be at least 99% and 94% confident that the OU process can improve the annual net profit for the Energy and the DJIA markets.

Also, we want to see if the OU process can improve the Sharpe ratio in the back-test. Our second null hypothesis is:

$$H_0: \quad ShapreRatio_1 > SharpeRatio_2$$

The *p-value* of the *t-test* are $1.39742278e-04 < 0.01$ and $0.0070 < 0.01$ respectively, and we can have at least 99% confidence to believe that the OU process indeed promotes the Sharpe ratio.

4.3.5. α-stable of Lévy

Considering that different α will produce different jumps, we aim to study the impact of a model's decision-making ability on investment portfolios. Consequently, we feed the reconstructed data into the SDELP-DDPG model, setting the parameter α to values of 1.2, 1.3, 1.4, 1.5, 1.6, and 1.7 for both the DJIA and Energy market. According to the results shown in Table 1, it is evident that the value of α exhibits a certain impact on the proposed model. The variation coefficient is a normalized measure of the dispersion of a probability distribution and is defined as the ratio of the standard deviation to the mean. It can be seen that the best α of the DJIA market is 1.7 about the Annual Return, and the best α of the Energy market is 1.4 about the Annual Return. For different stocks, the greater the data fluctuation, the smaller the α of the stock, which is in line with the nature of Lévy motion, that is, smaller α can produce larger jumps. We further can conclude from the results that the proposed model is not very sensitive to the hyperparameter α.

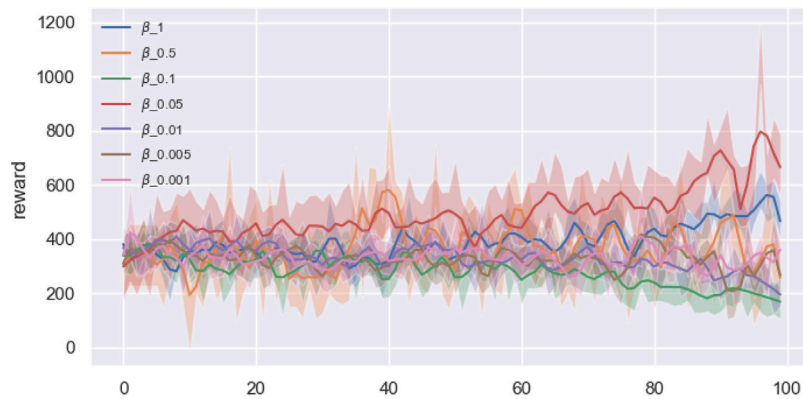

Fig. 7. Comparison of the reward Eq. (6) with different risk penalties (β) after training the SDELP-DDPG model 100 epochs on the Dow Jones Industrial Average market.

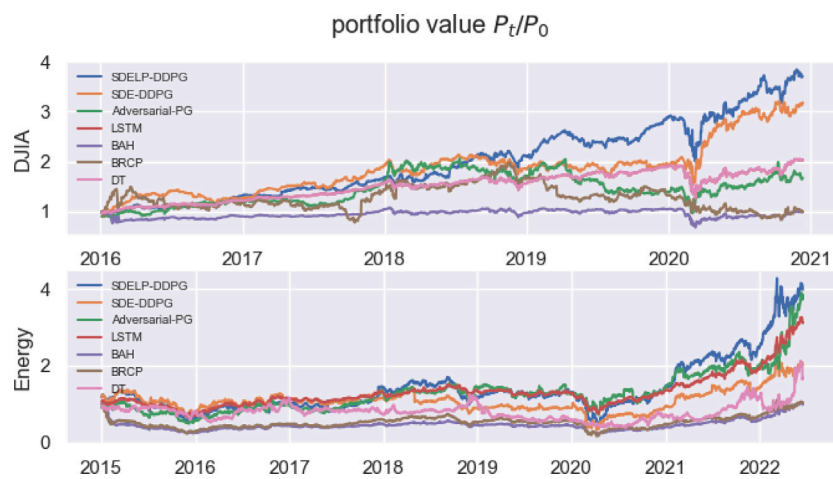

Fig. 8. Accumulated portfolio values ($APV, P_t/P_0$) over the interval of backtest period for the benchmark (the BAH and the UCRP) and CNN, LSTM, SDE-DDPG, SDELP-DDPG. The SDELP-DDPG consistently leads throughout the entire time span, experiencing only a few drawdown incidents.

Table 2
The evaluation results of different algorithms on DJIA,Energy, and Cryptocurrencies. In the Table, AR, AV and CRR represents the Annual Return, Annual Volatility and Cumulative Return Ratio, respectively.

Markets	Models	AR(%)	Sharpe	AV(%)	Sortino	MDD(%)	CRR
DJIA	SDELP-DDPG	**19.84**	**1.172**	25.30	**1.568**	31.63	**2.7613**
	SDE-Net (Kong, Sun, & Zhang, 2020)	18.27	1.066	24.84	1.300	**28.46**	2.2029
	DT (Wang et al., 2021)	12.54	0.853	**19.04**	0.958	33.91	1.0874
	Adversarial-PG (Liang et al., 2018)	12.00	0.491	31.29	0.600	53.14	0.8441
	LSTM (Yu, Si, Hu, & Zhang, 2019)	12.50	0.849	19.05	0.954	33.95	1.0816
	BAH	9.94	0.536	22.65	0.716	36.61	0.6173
	BCRP (Cover, 1991)	−8.31	−0.241	29.23	−0.301	58.18	−0.4317
Energy	SDELP-DDPG	**16.41**	0.626	43.29	**0.852**	74.45	**3.0667**
	SDE-Net (Kong et al., 2020)	12.44	0.428	42.07	0.529	76.75	1.0598
	DT (Wang et al., 2021)	13.20	0.386	50.73	0.581	69.65	0.7346
	Adversarial-PG (Liang et al., 2018)	12.84	0.452	41.63	0.581	78.88	1.2324
	LSTM (Yu et al., 2019)	16.16	**0.630**	42.00	0.844	**53.38**	3.0210
	BAH	11.91	0.514	**32.82**	0.693	60.77	1.4426
	BCRP (Cover, 1991)	13.06	0.461	41.66	0.591	77.60	1.2559

4.3.6. DDPG loss of SDE-Net driven by different motions

The research Yang et al. (2023) has already proven that $\alpha-$stable Lévy motion induced SDE-Net increases the accuracy of the model prediction. Furthermore, we demonstrate that the SDELP driven actor policy generator indeed enhances the stability and exploration ability of the model by plotting the loss of the DDPG algorithm in Fig. 9. In the DJIA market, the error in Eq. (17) of the critic network of the SDELP-DDPG model tends to be stable after 46 training epochs, while the SDE-DDPG model driven by Brownian motion does not exhibit stabilization. Similarly, the policy gradient of the non-Gaussian driven policy calculated by Eq. (16) is more stable than that of the Gaussian process driven model SDE-DDPG. Despite the high volatility in the Energy market, our SDELP-DDPG model's critic network error steadily stabilizes after 85 training epochs, and the policy gradient stabilizes after 50 epochs. In contrast, the SDE-DDPG model, which is driven by Brownian motion, encounters difficulties in stabilizing, even after undergoing 100 training sessions.

4.3.7. Performance comparison and computation time

We also compare the proposed model with SDE-Net (Kong et al., 2020) and commonly used models, such as the LSTM model (Yu et al.,

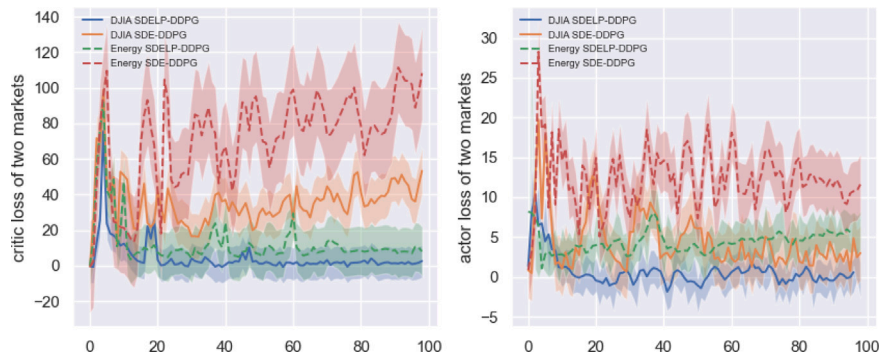

Fig. 9. The loss function of the critic network and the policy network on Dow Jones Industrial Average market and Energy market, updated by Eqs. (16) and (17) respectively. Both figures show the training process of the two networks driven by Lévy motion and Brownian motion over 100 epochs.

Table 3
The evaluation results of different algorithms on Cryptocurrency market. In the table, AR, AV and CRR represents the Annual Return, Annual Volatility and Cumulative Return Ratio, respectively.

Models	AR(%)	Sharpe	AV(%)	Sortino	MDD(%)	CRR
SDELP-DDPG	**58.37**	**1.114**	60.58	**1.482**	**51.33**	**6.7383**
SDE-Net (Kong et al., 2020)	50.24	1.103	68.43	1.479	59.26	6.6912
LSTM (Yu et al., 2019)	38.92	0.563	63.15	0.831	66.84	5.7080
BAH	34.55	0.543	**60.19**	0.701	73.08	4.1861
BCRP (Cover, 1991)	35.00	0.330	72.40	0.420	75.30	4.2730

2019) and the Adversarial-PG method (Liang et al., 2018). In addition, the Buy And Hold (BAH) and the Best Constant Rebalanced Portfolio (BCRP) are selected as the benchmark strategies. BCRP (Cover, 1991) is the optimal strategy in independent and identically distributed markets based on posterior observations of market data, and is often used as the performance target of new algorithms. The benchmark strategy does not have complex active position adjustment ideas, and is often used as the BAH and BCRP to compare with new strategies.

The outcomes of these methods are presented in Table 2. The data reveals that our proposed algorithm performs exceptionally well on three key metrics: annual return, Sortino ratio and CRR. Specifically, the Sharpe ratio of the SDELP-DDPG model in the energy market is only marginally lower by 0.04 points compared to the LSTM model, but it surpasses all other models in terms of Sharpe ratio. Within the framework of our proposed algorithm, the objective function accounts for returns, transaction costs, and a penalty term for relative entropy. As a result, the model shows promise in generating profits and managing risk. Moreover, the experimental results also confirm that our algorithm performs well on related metrics. However, when it comes to volatility and Maximum Drawdown (MDD), our approach does not show a distinct advantage, primarily because our objective function's design does not directly address these aspects of volatility and MDD.

The experimental results in the cryptocurrency market are shown in Table 3. When compared with other methods such as SDE-Net (Kong et al., 2020), LSTM (Yu et al., 2019), BAH, and BCRP (Cover, 1991), SDE
LP-DDPG demonstrates the best performance among them, excluding the AV. From 2015 to 2021, the cryptocurrency market witnessed substantial overall fluctuations. Nevertheless, the proposed model SDELP-DDPG manages to sustain high returns. This indicates that SDELP-DDPG can adapt to diverse scenarios characterized by sudden events and high volatility.

In Fig. 8, we plot and compare the accumulative portfolio value (APV) of various methods. It is evident that SDELP-DDPG shows a significant upward trend in APV, resulting in better performance for both the DJIA market and the Energy market.

To calculate the running time, multiple methods were executed over 100 trading periods on a computer equipped with an Intel Core i5-8250U CPU and 8 GB DDR3 2400MHz RAM. The average running times (in seconds) of SDELP-DDPG on the DJIA, Cryptocurrency, and Energy markets are 0.0567, 0.0115, and 0.0116 respectively. In comparison, the SDE-Net (Kong et al., 2020) has a slightly shorter average running time of 0.0102 on the Energy market, yet it takes longer (0.0569 and 0.0159) than the SDELP-DDPG model on the DJIA and Cryptocurrency markets. As for the LSTM (Yu et al., 2019), its average running times are 0.0590, 0.0176, and 0.0312 respectively. Consequently, the experimental results serve as compelling evidence for the practical viability of the proposed method in real-time portfolio management.

5. Conclusions and future work

In this paper, we propose a new reinforcement learning model to address the problem of financial portfolio management. It is designed to handle various input feature combinations and directly output portfolio weights as market behavior. Compared with previous portfolio management, we design our action policy generator with stochastic differential equations equipped with deep neural networks and Lévy motion, also design the critic with multilayer residual networks. We select actions based on the attention mechanism and the Ornstein–Uhlenbeck random process, and the hypothesis test is also given. Additionally, we define a risk-sensitive reward function based on cross-entropy to balance profit and risk. Furthermore, in order to verify the proposed model, we also use Modern Portfolio Theory to explore its maximum Sharpe Ratio and the minimum volatility. On the other hand, we explore its risk factors and the α stability of Lévy, and analyze the performance of our model from multiple perspectives. Experimental results demonstrate that the proposed model illustrates an outstanding performance.

While the proposed model has shown success in back-tests, there is still room for improvement in future works. One avenue for enhancement involves exploring various series of latent stochastic differential equations integrated with neural networks. Additionally, the application of the adjoint sensitivity method to stochastic differential equations is also worth further consideration. Furthermore, due to the convergence of the RL algorithm, we could only search for an optimal policy, rather than within the policy function set, which might also lead to the failure of finding a globally optimal strategy. The current online critic network can be adapted for the reinforcement learning agent to include an awareness of longer-term market reactions. Finally, although Lévy process-driven SDEs are advantageous for the proposed model, their drawbacks provide certain challenges for our future research.

Table A.4
Dow Jones Industrial Average markets.

ALCOA INC	GENERAL ELEC CO	JOHNSON&JOHNSON
GENERAL MOTORS	JP MORGAN CHASE	PROCTER & GAMBLE
COCA COLA CO	SBC COMMS	CITIGROUP
AT&T CORP	CATERPILLAR	HEWLETT-PACKARD
DU PONT CO	INTL BUS MACHINE	PHILIP MORRIS
INTEL CORP	MERCK & CO	EXXON MOBIL
MICROSOFT CP	AMER EXPRESS CO	BOEING CO
HOME DEPOT INC	HONEYWELL INTL	MCDONALDS CORP
3M COMPANY	UNITED TECH CP	WAL-MART STORES
WALT DISNEY CO	EASTMAN KODAK	INTL PAPER CO

Table A.5
Some of the top Cryptocurrencies ranked by market capitalization.

BITCOIN	DOGECOIN	ETHEREUM
LITECOIN	TETHER	NEM
STELLAR	MONERO	XRP

Since Lévy processes lack memory, they are unable to connect previous occurrences to future states, which makes them useless when historical data is important. Furthermore, Levy processes are limited in their capacity to depict financial time series that display interdependence due to the independence of increments. Therefore, overcoming these weaknesses of the Lévy process is also one of our future research directions.

CRediT authorship contribution statement

Zhen Huang: Methodology, Software, Original draft. **Junwei Duan:** Data curation, Visualization, Investigation. **Chuanlin Zhang:** Review, Editing. **Wenyong Gong:** Conceptualization, Funding acquisition, Supervision, Writing.

Declaration of competing interest

The authors declared that they have no conflicts of interest to this work. We declare that we do not have any commercial or associative interest that represents a conflict of interest in connection with the work submitted.

Acknowledgments

This work was supported in part by the National Nature Science Foundation of China under Grant T2341004, and in part by the Major projects of the National Social Science Foundation of China under Grant 24&ZD255, and in part by the Guangdong College Students' Innovation and Entrepreneurship Competition under Grant S202310559085.

Appendices

A.1. Markets for selected assets

See Tables A.4 and A.5.

A.2. Training and test data period

See Table A.6

A.3. The hypothesis test for the energy and the DJIA markets

See Tables A.7 and A.8

Table A.6
The data period of DJIA, Energy markets and cryptocurrencies.

Markets	Period	Time
DJIA	training	2009-01-02 to 2016-01-04
	testing	2016-01-05 to 2020-12-29
Energy	training	2000-01-04 to 2015-01-02
	testing	2015-01-03 to 2022-06-17
Cryptocurrencies	training	2015-08-08 to 2018-08-06
	testing	2018-08-07 to 2021-07-06

Table A.7
Comparison of trading performance with and without OU process for the Energy market.

Energy	ANP_1	$Sharp_1$	ANP_2	$Sharp_2$
1	0.1048	0.349	0.164	0.527
2	0.1075	0.362	0.1762	0.605
3	0.1115	0.389	0.2167	0.795
4	0.1406	0.539	0.1968	0.704
5	0.1336	0.494	0.1732	0.588
6	0.1093	0.377	0.1718	0.568
7	0.1080	0.374	0.1720	0.561
8	0.1300	0.476	0.1932	0.681
9	0.1280	0.468	0.1728	0.560
10	0.1359	0.508	0.1780	0.589
11	0.1247	0.450	0.1830	0.627
12	0.1257	0.457	0.1947	0.655
13	0.1207	0.435	0.1813	0.623
14	0.1553	0.621	0.1781	0.591
15	0.1312	0.481	0.1671	0.542
16	0.1451	0.568	0.1854	0.639
17	0.1095	0.379	0.1672	0.546
18	0.1161	0.407	0.1721	0.585
19	0.1451	0.568	0.1715	0.567
20	0.0651	0.403	0.1777	0.599
21	0.1225	0.437	0.1927	0.678
22	0.1300	0.476	0.1678	0.564
23	0.1429	0.551	0.1671	0.531
24	0.1242	0.453	0.1541	0.627
25	0.1319	0.496	0.1500	0.612

Table A.8
Comparison of trading performance with and without OU process for DJIA mraket.

Energy	ANP_1	$Sharp_1$	ANP_2	$Sharp_2$
1	0.1095	0.512	0.1329	0.68
2	0.0823	0.369	0.1243	0.635
3	0.1088	0.516	0.1496	0.767
4	0.0985	0.465	0.1265	0.661
5	0.1320	0.672	0.2233	1.349
6	0.1093	0.529	0.2454	0.450
7	0.0557	0.243	0.1417	0.717
8	0.0906	0.393	0.1993	1.236
9	0.1401	0.729	0.1553	0.874
10	0.1140	0.571	0.1681	0.950
11	0.1187	0.581	0.1316	0.678
12	0.1649	0.862	0.1146	0.552
13	0.1531	0.766	0.1254	0.650
14	0.1325	0.634	0.1829	1.068
15	0.1406	0.735	0.1423	0.734
16	0.0737	0.325	0.1472	0.751
17	0.0974	0.426	0.1741	0.885
18	0.0758	0.352	0.1634	0.909
19	0.0993	0.503	0.2088	1.210
20	0.0762	0.319	0.1802	1.044
21	0.0934	0.432	0.1678	1.039
22	0.0961	0.445	0.1653	0.905
23	0.1683	0.848	0.1340	0.732
24	0.1353	0.721	0.1444	0.811
25	0.0987	0.451	0.1413	0.713

Data availability

The data that has been used is confidential.

References

Almahdi, S., & Yang, S. Y. (2017). An adaptive portfolio trading system: A risk-return portfolio optimization using recurrent reinforcement learning with expected maximum drawdown. *Expert Systems with Applications*, *87*, 267–279.

Auh, J. K., & Cho, W. (2023). Factor-based portfolio optimization. *Economics Letters*, Article 111137.

Balduzzi, P., & Lynch, A. W. (1999). Transaction costs and predictability: Some utility cost calculations. *Journal of Financial Economics*, *52*(1), 47–78.

Becker, R., Clements, A. E., & McClelland, A. (2009). The jump component of S&P 500 volatility and the VIX index. *Journal of Banking & Finance*, *33*(6), 1033–1038.

Black, F., & Litterman, R. (1992). Global portfolio optimization. *Financial Analysts Journal*, 28–43.

Chaouki, A., Hardiman, S., Schmidt, C., Sérié, E., & De Lataillade, J. (2020). Deep deterministic portfolio optimization. *The Journal of Finance and Data Science*, *6*, 16–30.

Chen, T., Cheng, L., Liu, Y., Jia, W., & Ma, S. (2019). Incremental reinforcement learning—a new continuous reinforcement learning frame based on stochastic differential equation methods. arXiv preprint arXiv:1908.02974.

Costa, G., & Kwon, R. H. (2019). Risk parity portfolio optimization under a Markov regime-switching framework. *Quantitative Finance*, *19*(3), 453–471.

Cover, T. M. (1991). Universal portfolios. *Mathematical Finance*, *1*(1), 1–29.

Cover, T. M., & Ordentlich, E. (1996). Universal portfolios with side information. *Institute of Electrical and Electronics Engineers. Transactions on Information Theory*, *42*(2), 348–363.

Dabney, W., Ostrovski, G., & Barreto, A. (2020). Temporally-extended ϵ-greedy exploration. arXiv preprint arXiv:2006.01782.

Dewey, D. (2014). Reinforcement learning and the reward engineering principle. In *2014 AAAI spring symposium series*.

Fabbri, M., & Moro, G. (2018). Dow jones trading with deep learning: The unreasonable effectiveness of recurrent neural networks. In *Data* (pp. 142–153).

Fernholz, E. R., & Fernholz, E. R. (2002). *Stochastic portfolio theory*. Springer.

Filos, A. (2019). Reinforcement learning for portfolio management. arXiv preprint arXiv:1909.09571.

Gârleanu, N., & Pedersen, L. H. (2013). Dynamic trading with predictable returns and transaction costs. *The Journal of Finance*, *68*(6), 2309–2340.

Gunjan, A., & Bhattacharyya, S. (2023). A brief review of portfolio optimization techniques. *Artificial Intelligence Review*, *56*(5), 3847–3886.

Györfi, L., Lugosi, G., & Udina, F. (2006). Nonparametric kernel-based sequential investment strategies. *Mathematical Finance: An International Journal of Mathematics, Statistics and Financial Economics*, *16*(2), 337–357.

Han, Z., Hu, Y., & Lee, C. (2016). Optimal pricing barriers in a regulated market using reflected diffusion processes. *Quantitative Finance*, *16*(4), 639–647.

Heaton, J. B., Polson, N. G., & Witte, J. H. (2017). Deep learning for finance: deep portfolios. *Applied Stochastic Models in Business and Industry*, *33*(1), 3–12.

Helmbold, D. P., Schapire, R. E., Singer, Y., & Warmuth, M. K. (1998). On-line portfolio selection using multiplicative updates. *Mathematical Finance*, *8*(4), 325–347.

Hou, X., Wang, K., Zhong, C., & Wei, Z. (2021). St-trader: A spatial-temporal deep neural network for modeling stock market movement. *IEEE/CAA Journal of Automatica Sinica*, *8*(5), 1015–1024.

Huang, Z., Gong, W., & Duan, J. (2023). TBDQN: A novel two-branch deep Q-network for crude oil and natural gas futures trading. *Applied Energy*, *347*, Article 121321.

Huang, Z., Li, N., Mei, W., & Gong, W. (2023). Algorithmic trading using combinational rule vector and deep reinforcement learning. *Applied Soft Computing*, Article 110802.

Huang, W., Wang, Y., & Yi, X. (2017). A deep reinforcement learning approach to preserve connectivity for multi-robot systems. In *2017 10th international congress on image and signal processing, bioMedical engineering and informatics* (pp. 1–7). IEEE.

Isichenko, M. (2021). *Quantitative portfolio management: The art and science of statistical arbitrage*. John Wiley & Sons.

Jiang, Z., Xu, D., & Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. arXiv preprint arXiv:1706.10059.

Kanwar, N. (2019). *Deep reinforcement learning-based portfolio management*. The University of Texas at Arlington.

Ken-Iti, S. (1999). *Lévy processes and infinitely divisible distributions*: *vol. 68*, Cambridge University Press.

Kolm, P. N., Tütüncü, R., & Fabozzi, F. J. (2014). 60 years of portfolio optimization: Practical challenges and current trends. *European Journal of Operational Research*, *234*(2), 356–371.

Kong, L., Sun, J., & Zhang, C. (2020). Sde-net: Equipping deep neural networks with uncertainty estimates. arXiv preprint arXiv:2008.10546.

Kurumatani, K. (2020). Time series forecasting of agricultural product prices based on recurrent neural networks and its evaluation method. *SN Applied Sciences*, *2*(8), 1434.

Lai, Z.-R., Dai, D.-Q., Ren, C.-X., & Huang, K.-K. (2017). A peak price tracking-based learning system for portfolio selection. *IEEE Transactions on Neural Networks and Learning Systems*, *29*(7), 2823–2832.

Lai, Z.-R., Tan, L., Wu, X., & Fang, L. (2020). Loss control with rank-one covariance estimate for short-term portfolio optimization. *Journal of Machine Learning Research*, *21*(1), 3815–3851.

Li, B., & Hoi, S. C. (2012). On-line portfolio selection with moving average reversion. arXiv preprint arXiv:1206.4626.

Li, C., Wu, Y., Lu, Z., Wang, J., & Hu, Y. (2020). A multiperiod multiobjective portfolio selection model with fuzzy random returns for large scale securities data. *IEEE Transactions on Fuzzy Systems*, *29*(1), 59–74.

Li, B., Zhao, P., Hoi, S. C., & Gopalkrishnan, V. (2012). PAMR: Passive aggressive mean reversion strategy for portfolio selection. *Machine Learning*, *87*, 221–258.

Liang, Z., Chen, H., Zhu, J., Jiang, K., & Li, Y. (2018). Adversarial deep reinforcement learning in portfolio management. arXiv preprint arXiv:1808.09940.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., et al. (2015). Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971.

Lintner, J. (1975). The valuation of risk assets and the selection of risky investments in stock portfolios and capital budgets. In *Stochastic optimization models in finance* (pp. 131–155). Elsevier.

Markovitz, H. M. (1959). *Portfolio selection: Efficient diversification of investments*. John Wiley.

Markowitz, H. M. (1952). Portfolio selection. *The Journal of Finance*, *7*(1), 77.

Medsker, L. R., & Jain, L. (2001). Recurrent neural networks. *Design and Applications*, *5*(64–67), 2.

Memory, L. S.-T. (2010). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Merton, R. C. (1976). Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, *3*(1–2), 125–144.

Moody, J., Wu, L., Liao, Y., & Saffell, M. (1998). Performance functions and reinforcement learning for trading systems and portfolios. *Journal of Forecasting*, *17*(5–6), 441–470.

Mossin, J. (1966). Equilibrium in a capital asset market. *Econometrica*, 768–783.

Naseri, R., & Malek, A. (2014). Numerical optimal control for problems with random forced spde constraints. *International Scholarly Research Notices*, *2014*.

Quer, J., & Borrell, E. R. (2022). Connecting stochastic optimal control and reinforcement learning. arXiv preprint arXiv:2211.02474.

Renshaw, E. (1987). The discrete uhlenbeck–ornstein process. *Journal of Applied Probability*, *24*(4), 908–917.

Roncalli, T., & Weisang, G. (2016). Risk parity portfolios with risk factors. *Quantitative Finance*, *16*(3), 377–388.

Sharp, K. P. (1990). Stochastic differential equations in finance. *Applied Mathematics and Computation*, *37*(2), 131–148.

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. In *International conference on machine learning* (pp. 387–395). Pmlr.

Song, D. R., Yang, C., McGreavy, C., & Li, Z. (2018). Recurrent deterministic policy gradient method for bipedal locomotion on rough terrain challenge. In *2018 15th international conference on control, automation, robotics and vision* (pp. 311–318). IEEE.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT Press.

Topaloglou, N., Vladimirou, H., & Zenios, S. A. (2008). A dynamic stochastic programming model for international portfolio management. *European Journal of Operational Research*, *185*(3), 1501–1524.

Treynor, J. L. (1961). Market value, time, and risk. *Time, and Risk*.

Wang, Z., Huang, B., Tu, S., Zhang, K., & Xu, L. (2021). DeepTrader: a deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding. In *Proceedings of the AAAI conference on artificial intelligence, vol. 35, no. 1* (pp. 643–650).

Yang, L., Gao, T., Lu, Y., Duan, J., & Liu, T. (2023). Neural network stochastic differential equation models with applications to financial data forecasting. *Applied Mathematical Modelling*, *115*, 279–299.

Yang, H., Liu, X.-Y., Zhong, S., & Walid, A. (2020). Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the first ACM international conference on AI in finance* (pp. 1–8).

Ye, Y., Pei, H., Wang, B., Chen, P.-Y., Zhu, Y., Xiao, J., et al. (2020). Reinforcement-learning based portfolio management with augmented asset movement prediction states. In *Proceedings of the AAAI conference on artificial intelligence, vol. 34, no. 01* (pp. 1112–1119).

Yu, Y., Si, X., Hu, C., & Zhang, J. (2019). A review of recurrent neural networks: LSTM cells and network architectures. *Neural Computation*, *31*(7), 1235–1270.

Yuan, G., Ding, D., Duan, J., Lu, W., & Wu, F. (2022). Total value adjustment of bermudan option valuation under pure jump Lévy fluctuations. *Chaos. An Interdisciplinary Journal of Nonlinear Science*, *32*(2).

Zhu, Q. (2014). Asymptotic stability in the pth moment for stochastic differential equations with Lévy noise. *Journal of Mathematical Analysis and Applications*, *416*(1), 126–142.